Giancarlo Fedolfi

+1 (860) 967-6556 | fedolficarlo@gmail.com | LinkedIn | GitHub | Website

EDUCATION

Wesleyan University | Middletown, CT **Expected Graduation: May 2027**

B.A. in Computer Science, German | GPA: 3.74/4.0

- **Relevant Coursework:** Software Engineering, Automata Theory, Discrete Mathematics, Logic & Functional Programming, Computational Media (Game Development), Data Structures, Information Security and Privacy
- **Honors:** Dean's List, Scholar-Athlete Honors Award
- **Activities:** Varsity Indoor and Outdoor Track and Field Captain

TECHNICAL SKILLS

Languages: Python, C++, C, Go, JavaScript/TypeScript, SQL, PHP, C#, SMLNJ, GDScript
Frameworks & Tools: React, Go (chi), Django, Node.js, PostgreSQL, Docker, Git, GitHub Actions, MongoDB, Vite, Railway, Vercel, Figma, LiteLLM, BeautifulSoup, JIRA

RELEVANT EXPERIENCE

turingpoint GmbH | Hamburg, Germany **September 2025 - December 2025**

Software Engineer Intern

- Built a web-based CVE classification system in Python using LiteLLM to route Anthropic API calls, cutting manual research time by 85%.
- Automated data ingestion by integrating the Brave Search API with BeautifulSoup scraping.
- Designed and iterated on UI/UX mockups in Figma with engineers and product managers to make the tool usable by non-technical reviewers.

RESEARCH

Bioinformatics Research | Wesleyan University - Prof. Cohan, Prof Krizanc **February 2026 - Present**

Research Assistant

- Analyzed genome sequencing data to separate species-level genomes using Python and Pandas.
- Built a pipeline to gather data from the web, feeding it into Bakta, Roary, Veryfasttree, then Ecosim to get gene presence/absence trees, replacing a manual process and cutting runtime by half
- Visualized gene presence/absence trees with Matplotlib.

Additive Manufacturing Research | Wesleyan University - Prof. Chang-Davidson **June 2026 - Present**

Research Assistant

- Developed a model using graph theory to simulate heat dispersion in a DED metal print.
- Simulated and printed different geometries to test accuracy and calibrate the software.

PROJECTS

SolverRL | C++17, pybind11, Python, SWI-Prolog, Stable-Baselines3, CMake, GoogleTest **August 2026**

- Reproduced a research pipeline that distills a frozen PPO teacher into an executable SWI-Prolog decision list over a finite KeyDoor MDP (~16K reachable states): reachable-state census, FOIL-style relational rule induction in C++, and first-match Prolog emission with cut semantics.
- Built a certified EXPAND loop in C++ that accepts program edits only when exact discounted return rises (Bellman linear solves over the full model), reports monotone certified-return curves and an advantage-gap certificate vs. the greedy teacher, and targets paper-comparable R1 optimum / R2 recovery from undertrained teachers.
- Extended the same C++ rule-learning core to MiniGrid DoorKey relational transfer (8×8 train → 6×6 / 16×16 eval with unchanged Prolog) and a thin CartPole DAgger honesty baseline; wired pybind11 + pytest/GoogleTest CI and one-command reproduction from checkpoint → census → distill → expand → metrics.

Remindr | C++17, Go, React, PostgreSQL, Docker, GitHub Actions **June 2026**

- Extended a cross-platform desktop reminder daemon (C++17) with offline-first cloud sync: background worker pushes/pulls via REST, Unix-socket IPC, and a last-write-wins merge with soft-delete tombstones.
- Built a Go REST API (JWT auth, CRUD, sync endpoint) backed by PostgreSQL (Neon); deployed on Railway with Docker; React dashboard on Vercel with custom domain and CORS.
- Implemented libcurl sync client in the daemon, credential persistence for launchd, and GitHub Actions CI (Go tests, web build, C++/Linux + macOS).

TermiTama | C++17, CMake, FTXUI, JSON **April 2026**

- Built a terminal virtual pet game in C++17 using FTXUI with an FSM-driven lifecycle (Baby → Adult), four stat systems with offline time-delta decay capped at 12 hours, and a genetics engine with heritable stat caps and ±10% random mutation across breeding generations.
- Implemented a persistent save system using JSON serialization that auto-saves after every action, restores full game state on launch, and supports a Pet Box of up to 6 simultaneously active pets with free switching and adoption.